- Each response stream creates a new thread and copies over the thread local variables from the original thread. Having too many thread local variables can negatively impact performance. Similarly, a large number of threads can also hinder performance.
- Failing to close the response stream will leave the corresponding socket open forever. Make sure to call `close` whenever you are using a response stream.
- WEBrick servers buffer all responses, and so including `ActionController::Live` will not work. You must use a web server which does not automatically buffer responses.

13 Log Filtering

Rails keeps a log file for each environment in the `log` folder. These are extremely useful when debugging what's actually going on in your application, but in a live application you may not want every bit of information to be stored in the log file.

13.1 Parameters Filtering

You can filter out sensitive request parameters from your log files by appending them to `config.filter_parameters` in the application configuration. These parameters will be marked [FILTERED] in the log.

```
config.filter_parameters << :password
```

13.2 Redirects Filtering

Sometimes it's desirable to filter out from log files some sensitive locations your application is redirecting to. You can do that by using the `config.filter_redirect` configuration option:

```
config.filter_redirect << 's3.amazonaws.com'
```

You can set it to a String, a Regexp, or an array of both.

```
config.filter_redirect.concat ['s3.amazonaws.com', /private_path/]
```

Matching URLs will be marked as '[FILTERED]'.

14 Rescue

Most likely your application is going to contain bugs or otherwise throw an exception that needs to be handled. For example, if the user follows a link to a resource that no longer exists in the database, Active Record will throw the `ActiveRecord::RecordNotFound` exception.

Rails' default exception handling displays a "500 Server Error" message for all exceptions. If the request was made locally, a nice traceback and some added information gets displayed so you can figure out what went wrong and deal with it. If the request was remote Rails will just display a simple "500 Server Error" message to the user, or a "404 Not Found" if there was a routing error or a record could not be found. Sometimes you might want to customize how these errors are caught and how they're displayed to the user. There are several levels of exception handling available in a Rails application:

14.1 The Default 500 and 404 Templates

By default a production application will render either a 404 or a 500 error message. These messages are contained in static HTML files in the `public` folder, in `404.html` and `500.html` respectively. You can customize these files to add some extra information and layout, but remember that they are static; i.e. you can't use RHTML or layouts in them, just plain HTML.